MÉTODOS ITERATIVOS PARA SISTEMAS DE ECUACIONES LINEALES

Cael Mathew Cuevas Alconini

C.I. 10066280

Universidad Mayor de San Andrés

Facultad de Ciencias Puras y Naturales

Asignatura: Métodos Numéricos

Docente: Lic. Brígida Carvajal

Acceso al repositorio: https://github.com/Kaelthas153/MetodosNumericos

16 de marzo de 2026

Tarea 1 Fundamentación teórica

1. Método de Gauss-Seidel

El método de **Gauss-Seidel** es un procedimiento iterativo para resolver sistemas de ecuaciones lineales de la forma $Ax = b$. Se considera una mejora del método de Jacobi, ya que utiliza los valores actualizados de las variables tan pronto como están disponibles.

Descripción del Algoritmo

El método descompone la matriz $A\ en\ A = L + D + U$, donde L es una matriz triangular inferior, D es la diagonal y U es una matriz triangular superior. La ecuación iterativa se define como:

$$(D + L)x^{(k+1)} = b - Ux^{(k)}$$

En términos de sus componentes individuales, para cada fila i, la fórmula es:

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k)}\right)$$

Condiciones de Convergencia

El método de Gauss-Seidel converge a la solución única del sistema si se cumple al menos una de las siguientes condiciones:

1. **Matriz Diagonalmente Dominante:** La matriz A es estrictamente dominante por filas ($|a_{ii}| > \Sigma_{j\neq i}|a_{ij}|$).
2. **Matriz Definida Positiva:** La matriz A es simétrica y definida positiva.

Principales Características

- **Eficiencia de memoria:** No requiere almacenar una copia separada del vector de la iteración anterior (a diferencia de Jacobi).
- **Velocidad:** Generalmente converge el doble de rápido que el método de Jacobi.
- **Secuencialidad:** Debido a que $x_i^{(k+1)}$ depende de valores ya calculados en la misma iteración, su paralelización es más compleja que otros métodos.

2. Sobre-relajación Sucesiva (SOR)

El método **SOR** (*Successive Over-Relaxation*) es una variante del método de Gauss-Seidel diseñada para acelerar la convergencia mediante el uso de un parámetro de relajación ω.

Descripción del Algoritmo

El algoritmo calcula una solución intermedia basada en Gauss-Seidel y luego aplica un promedio ponderado con la iteración anterior. La fórmula por componentes es:

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}}\left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k)}\right)$$

Donde ω es el **factor de relajación**.

Condiciones de Convergencia

- **Teorema de Kahan:** El método solo puede converger si $0 < \omega < 2$.
- Si la matriz A es simétrica y definida positiva, el método converge para cualquier $\omega \in (0, 2)$.
- $\omega = 1$ **:** El método se reduce exactamente a Gauss-Seidel.
- $\omega > 1$ **:** Se denomina *sobre-relajación* y se usa para acelerar la convergencia de sistemas que convergen lentamente.

Principales Características

- **Optimización:** La clave del éxito de SOR radica en encontrar el ω **óptimo**, lo cual puede reducir drásticamente el número de iteraciones.
- **Sensibilidad:** Un valor de ω inadecuado puede hacer que el método sea más lento que Gauss-Seidel o incluso que diverja.

3. Método del Gradiente Conjugado (CG)

El **Gradiente Conjugado** es un algoritmo para la solución numérica de sistemas de ecuaciones lineales cuyas matrices son simétricas y definidas positivas. A diferencia de los anteriores, es un método de búsqueda en subespacios de Krylov.

Descripción del Algoritmo

En lugar de despejar variables, el CG ve el problema como la minimización de la función cuadrática:

$$f(x) = \frac{1}{2}x^T Ax - b^T x$$

El algoritmo genera una secuencia de vectores de dirección p_k que son **conjugados** respecto a A (es decir, $p_i^T A p_j = 0$). Los pasos principales por iteración son:

1. Calcular el residuo $r_k = b - Ax_k$.
2. Actualizar la dirección de búsqueda p_k basándose en el residuo actual y la dirección anterior.
3. Calcular el tamaño del paso a_k que minimiza la función en esa dirección.
4. Actualizar la solución: $x_{k+1} = x_k + \alpha_k p_k$.

Condiciones de Convergencia

- La matriz A debe ser **Simétrica y Definida Positiva (SPD)**.
- Teóricamente, el método converge a la solución exacta en un máximo de n iteraciones (donde n es el tamaño del sistema), aunque en la práctica se usa como método iterativo para obtener una aproximación en mucho menos pasos.

Principales Características

- **Memoria:** Es muy eficiente para matrices dispersas de gran tamaño, ya que solo requiere el resultado del producto matriz-vector Ax.
- **Superioridad:** Para sistemas grandes y bien condicionados, es significativamente más rápido que SOR o Gauss-Seidel.
- **Sin parámetros:** No requiere que el usuario elija un factor de relajación como en SOR.

Resumen comparativo

Característica	**Gauss-Seidel**	**SOR**	**Gradiente Conjugado**
Tipo de Método	Estacionario	Estacionario (acelerado)	No estacionario (Krylov)

Requisito Matriz	Diagonal dominante / SPD	$0 < \omega < 2/SPD$	Simétrica y Definida Positiva
Parámetros	Ninguno	Factor ω	Ninguno
Uso Ideal	Sistemas pequeños	Sistemas donde se conoce ω	Sistemas grandes y dispersos

Tarea 2: Ejemplo Introductorio y Desarrollo de Métodos

Para este caso de prueba, utilizaremos un sistema de ecuaciones lineales

$$Ax = b$$

que es estrictamente dominante por diagonal para asegurar la convergencia.

Planteamiento del Sistema

Definimos el sistema con una solución exacta conocida:

$$x = [1, 2, -1]^T$$

$$\begin{cases} 10x_1 - x_2 + 2x_3 = 6 \\ -x_1 + 11x_2 - x_3 = 25 \\ 2x_1 - x_2 + 10x_3 = -11 \end{cases}$$

1. Método de Gauss-Seidel

Vector inicial:

$$x^{(0)} = [0, 0, 0]^T$$

.

Iteración 1:

- $x_1^{(1)} = \dfrac{6 + (0) - 2(0)}{10} = 0.6$
- $x_2^{(1)} = \dfrac{25 + (0.6) + (0)}{11} = 2.3273$
- $x_3^{(1)} = \dfrac{-11 - 2(0.6) + (2.3273)}{10} = -0.9873$

Iteración 2:

- $x_1^{(2)} = \dfrac{6 + (2.3273) - 2(-0.9873)}{10} = 1.0302$
- $x_2^{(2)} = \dfrac{25 + (1.0302) + (-0.9873)}{11} = 2.2766$
- $x_3^{(2)} = \dfrac{-11 - 2(1.0302) + (2.2766)}{10} = -1.0784$

2. Método de Sobre-relajación Sucesiva (SOR)

Utilizaremos un factor de relajación

$\omega = 1.05$
y el vector inicial

$x^{(0)} = [0, 0, 0]^T$
.

Iteración 1:

- $x_1^{(1)} = (1 - 1.05)(0) + 1.05(0.6) = 0.63$
- $x_2^{(1)} = (1 - 1.05)(0) + 1.05(\dfrac{25 + 0.63 + 0}{11}) = 2.4457$
- $x_3^{(1)} = (1 - 1.05)(0) + 1.05(\dfrac{-11 - 2(0.63) + 2.4457}{10}) = -1.0305$

Iteración 2:

- $x_{1,GS} = \dfrac{6 + 2.4457 - 2(-1.0305)}{10} = 1.0507 \rightarrow x_1^{(2)} = (1 - 1.05)(0.63) + 1.05(1.0507) = 1.0717$
- $x_{2,GS} = \dfrac{25 + 1.0717 + (-1.0305)}{11} = 2.2765 \rightarrow x_2^{(2)} = (1 - 1.05)(2.4457) + 1.05(2.2765) = 2.2680$
- $x_{3,GS} = \dfrac{-11 - 2(1.0717) + 2.2680}{10} = -1.0875 \rightarrow x_3^{(2)} = (1 - 1.05)(-1.0305) + 1.05(-1.0875) = -1.0903$

3. Método del Gradiente Conjugado (CG)

Vector inicial:

$x_0 = [0, 0, 0]^T$

.

Iteración 1:

1. Residuo
   $r_0 = b - Ax_0 = [6, 25, -11]^T$

   .
2. Dirección
   $p_0 = r_0 = [6, 25, -11]^T$

   .
3. Escalar
   $$\alpha_0 = \frac{r_0^T r_0}{p_0^T A p_0} = \frac{782}{8321} \approx 0.0939$$

   .
4. Actualizar
   $x_1 = x_0 + \alpha_0 p_0 = [0.5634, 2.3475, -1.0329]^T$

   .

Iteración 2:

1. Nuevo residuo
   $r_1 = r_0 - \alpha_0 A p_0 = [4.779, -1.292, -0.389]^T$

   .
2. Escalar
   $$\beta_0 = \frac{r_1^T r_1}{r_0^T r_0} = \frac{24.658}{782} \approx 0.0315$$

   .
3. Nueva dirección
   $p_1 = r_1 + \beta_0 p_0 = [4.968, -0.504, -0.735]^T$

   .
4. Nuevo paso
   $$\alpha_1 = \frac{r_1^T r_1}{p_1^T A p_1} \approx 0.0526$$

   .
5. Actualizar
   $x_2 = x_1 + \alpha_1 p_1 = [0.8247, 2.3210, -1.0716]^T$

   .

Comparación con la Solución Exacta (

$x = [1, 2, -1]^T$

)

Tras solo 2 iteraciones, se observa que:

- **Gauss-Seidel** y **SOR** se aproximan rápidamente debido a la dominancia diagonal.
- **Gradiente Conjugado** muestra una reducción drástica del residuo, perfilándose para converger en la tercera iteración.

Tarea 3 Aplicación de los tres métodos al ejemplo introductorio

Bloque de código (Python)

```
import numpy as np

# Configuración del sistema de la Tarea 2
A = np.array([[10, -1, 2],
              [-1, 11, -1],
              [2, -1, 10]], dtype=float)
b = np.array([6, 25, -11], dtype=float)
x_exacto = np.array([1, 2, -1], dtype=float)
x0 = np.zeros(len(b))
iteraciones = 3

def imprimir_paso(metodo, i, x):
    print(f"{metodo} - Iteración {i}: {x}")

# --- 1. GAUSS-SEIDEL ---
print("--- Ejecutando Gauss-Seidel ---")
x_gs = x0.copy()
for i in range(1, iteraciones + 1):
    for j in range(len(b)):
        suma = b[j] - np.dot(A[j, :j], x_gs[:j]) - np.dot(A[j, j+1:],
x_gs[j+1:])
        x_gs[j] = suma / A[j, j]
    imprimir_paso("GS", i, x_gs.copy())

# --- 2. SOR (w = 1.05) ---
print("\n--- Ejecutando SOR (w=1.05) ---")
x_sor = x0.copy()
w = 1.05
for i in range(1, iteraciones + 1):
    for j in range(len(b)):
        suma = b[j] - np.dot(A[j, :j], x_sor[:j]) - np.dot(A[j, j+1:],
x_sor[j+1:])
        x_gs_val = suma / A[j, j]
        x_sor[j] = (1 - w) * x_sor[j] + w * x_gs_val
    imprimir_paso("SOR", i, x_sor.copy())
```

```python
# --- 3. GRADIENTE CONJUGADO ---
print("\n--- Ejecutando Gradiente Conjugado ---")
x_cg = x0.copy()
r = b - np.dot(A, x_cg)
p = r.copy()
for i in range(1, iteraciones + 1):
    alpha = np.dot(r, r) / np.dot(p, np.dot(A, p))
    x_cg = x_cg + alpha * p
    r_nuevo = r - alpha * np.dot(A, p)
    beta = np.dot(r_nuevo, r_nuevo) / np.dot(r, r)
    p = r_nuevo + beta * p
    r = r_nuevo
    imprimir_paso("CG", i, x_cg.copy())

print("\n--- Comparación Final ---")
print(f"Solución Exacta: {x_exacto}")
```

Captura de Corrida

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
--- Ejecutando Gauss-Seidel ---
GS - Iteración 1: [ 0.6         2.32727273 -0.98727273]
GS - Iteración 2: [ 1.03018182  2.2766281  -1.07837355]
GS - Iteración 3: [ 1.04333752  2.26954218 -1.08171329]

--- Ejecutando SOR (w=1.05) ---
SOR - Iteración 1: [ 0.63       2.4465    -1.0304175]
SOR - Iteración 2: [ 1.07177018  2.26798594 -1.09041234]
SOR - Iteración 3: [ 1.04353661  2.26848984 -1.08143064]

--- Ejecutando Gradiente Conjugado ---
CG - Iteración 1: [ 0.55651761  2.31882339 -1.02028229]
CG - Iteración 2: [ 1.03291869  2.29963408 -1.01361393]
CG - Iteración 3: [ 1.04326923  2.26923077 -1.08173077]

--- Comparación Final ---
Solución Exacta: [ 1.  2. -1.]
```

Tarea 4 Implementación en Excel

MÉTODO: GAUSS-SEIDEL				
Iteración	x1	x2	x3	Error (Norma Inf)
0	0	0	0	-
1	0.6	2.327.273	-987.273	0.4
2	1.030.182	2.276.628	-1.078.374	2.766.281
3	1.043.338	2.269.542	-1.081.713	26.954.218
4	1.043.297	2.269.235	-1.081.736	26.923.487
5	1.043.271	226.923	-1.081.731	26.923.043

MÉTODO: SOR (w=1.05)				
Iteración	x1	x2	x3	Error (Norma Inf)
0	0	0	0	-
1	0.63	2.4465	-10.304	4.465
2	1.0718	2.268	-10.904	26.798.594
3	1.0435	2.2685	-10.814	26.848.984
4	1.0431	2.2693	-10.817	26.928.175
5	1.0433	2.2692	-10.817	26.923.118

MÉTODO: GRADIENTE CONJUGADO				
Iteración	x1	x2	x3	Error (Norma Inf)
0	0	0	0	-
1	556.518	2.318.823	-1.020.282	44.348.239
2	1.032.919	2.299.634	-1.013.614	29.963.408
3	1.043.269	2.269.231	-1.081.731	26.923.077

Bibliografía

- **Burden, R. L., & Faires, J. D. (2011).** *Análisis Numérico* (9.ª ed.). Cengage Learning.
  Utilizada para la fundamentación teórica del método de Gauss-Seidel y las condiciones de convergencia basadas en matrices diagonalmente dominantes.
- **Chapra, S. C., & Canale, R. P. (2015).** *Métodos Numéricos para Ingenieros* (7.ª ed.). McGraw-Hill Education.
  Referenciada para la descripción del método de Sobre-relajación Sucesiva (SOR), la interpretación del factor ω y la implementación práctica de algoritmos iterativos.
- **Saad, Y. (2003).** *Iterative Methods for Sparse Linear Systems* (2.ª ed.). Society for Industrial and Applied Mathematics (SIAM).
  Consultada para la fundamentación avanzada del Método del Gradiente Conjugado y el estudio de los subespacios de Krylov.